# Episode Structure — Walkthrough Script

*Keyed to numbered steps in the diagram. Use as a verbal guide while sketching the corresponding boxes.*

### Step 1 — Episode Start

An episode is one complete training example. At the start, we sample a random waveform from a pool of seven initial conditions — Gaussian pulses, Mexican hat wavelets, solitons, and others, including waveforms with negative values. The solver initializes a level-1 mesh: starting from 4 base elements, each is refined once, giving 8 elements. The initial condition is then projected onto this mesh. Starting at level 1 means the agent can coarsen right from the beginning — it has room to manage resources in both directions.

### Steps 2–3 — Error Indicators and Thresholds

The episode is divided into $N_{\text{remesh}} = 4$ remesh intervals. Each interval starts by computing an error indicator $e_k$ for every active element on the current mesh. From the error distribution, two classification thresholds are set:

$$e_{\max} = \alpha \cdot \|e\|_\infty \qquad \text{(refinement threshold)}$$
$$e_{\min} = e_{\max}^{\beta} \qquad \text{(coarsening threshold, with } \beta = 1.2\text{)}$$

These are computed once and held fixed for all adaptation rounds within this remesh interval. Error indicators will be recomputed each round as the mesh changes, but the classification targets don't move. This creates three zones: elements above $e_{\max}$ are candidates for refinement, elements below $e_{\min}$ are candidates for coarsening, and elements in between are in the neutral zone.

The details of how $e_k$ is computed are covered in the observation space description (→ Component 4).

### Step 4 — Queue Assembly

Each remesh interval contains `max_level` = 3 adaptation rounds. At the start of each round, error indicators are recomputed on the current mesh, and a priority queue is assembled from all active elements.

*Pause here — switch to the queue mechanics board.* (→ Component 5: Queue Mechanics)

### Steps 5–6 — Element Visit (the core RL step)

The agent is presented with one element at a time, in queue order. For each element:

1. **Observe:** The environment constructs an 8-component observation vector from the current mesh state.
   *Pause — switch to the observation space board.* (→ Component 4: Observation Space)
2. **Mask:** The environment computes which of the three actions (refine, coarsen, do-nothing) are structurally valid. For example: can't refine past `max_level`; can't coarsen if it would violate 2:1 balance or if the sibling isn't active. Do-nothing is always valid. Invalid actions are masked so the agent cannot select them.
3. **Decide:** The policy network receives the observation and mask, and outputs a probability distribution over valid actions. An action is sampled.
4. **Execute:** The solver performs the mesh operation. If the action creates a 2:1 balance violation, cascading refinements are triggered. Cascade-consumed elements are removed from the remaining queue for this round.

5. **Local reward:** A classification-based shaping reward is computed based on whether the action was appropriate for this element's error level.
   *Pause — switch to the reward structure board.* (→ Component 6: Dual Reward)

This repeats for every element in the queue. The observation is always built fresh from the current mesh state — later elements see the consequences of earlier actions within the same round.

### Step 7 — Multi-Level Refinement Across Rounds

After all elements have been visited, the round is complete. The queue is rebuilt from the updated mesh. This is how multi-level refinement emerges naturally: elements created by refinement in round 1 appear in round 2's queue and can be refined further.

- Round 1: level $0 \to 1$
- Round 2: level $1 \to 2$
- Round 3: level $2 \to 3$

A round of all do-nothings is also meaningful — the agent has learned to recognize a well-adapted mesh.

### Steps 8–9 — Solver Advance and Global Reward

After all `max_level` rounds, the adaptation phase is over. The solver advances the PDE forward by time $T$, taking multiple CFL-limited sub-steps internally. During this advance, element-wise maximum errors are tracked:

$$\hat{e}_k = \max_{t \in [t_\tau,\, t_\tau + T]} e_k(t)$$

This captures transient high-error events that instantaneous error at $t_\tau + T$ would miss — for example, a wave pulse that passes through an element during the interval but has moved on by the time the interval ends.

A global retrospective reward is then computed from these max-over-interval errors, assessing the overall quality of the adapted mesh during the interval. This reward is delivered on the final RL step of the interval and propagated backward through all preceding element visits via the RL algorithm's credit assignment mechanism.

*Both the local and global rewards are explained in detail on the reward board.* (→ Component 6: Dual Reward)

### Step 10 — Episode End

After $N_{\text{remesh}} = 4$ remesh intervals, the episode ends. The agent has made approximately $4 \times 3 \times 15 \approx 180$ decisions. The RL algorithm uses the full trajectory to update the policy network, and a new episode begins with a fresh waveform.

### Key Points

1. The multi-round structure is what enables multi-level refinement — each round can take elements one level deeper, up to `max_level`.
2. Thresholds are fixed per remesh interval; error indicators are recomputed per round. The agent chases a stationary classification target within each interval.
3. The agent is not involved during the PDE advance. Adaptation and solver phases are strictly separated within each remesh interval.